

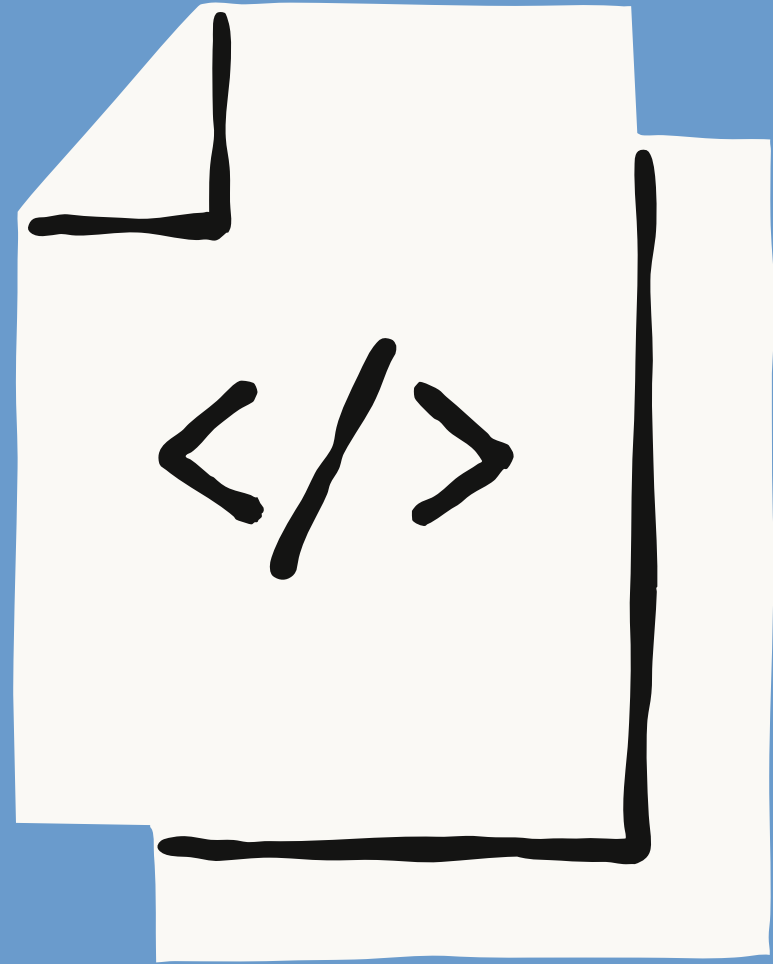
Building Effective AI Agents: Architecture Patterns and Implementation Frameworks

Accelerate your enterprise AI transformation
with proven strategies from Anthropic's
customers and internal teams.



Contents

Executive summary	4
Common use cases and applications for AI agents	7
Common architecture patterns	10
Looking forward: the future of building AI agents	27
Next steps	29



Chapter 1

Executive summary

Executive summary

Generative AI answers questions. AI agents solve problems.

For companies across industries, agents offer the potential for scaling operations in ways current automation could never deliver: open-ended problem-solving, dynamic decision-making, and complex multistep processes where the path forward isn't predetermined.

Organizations are seeing significant results from autonomous agents in production. For example, Coinbase, the leading cryptocurrency exchange managing \$226 billion in quarterly trading volume, built agentic customer support systems powered by Claude. Their Claude-powered agents handle thousands of messages per hour while maintaining 99.99% availability—critical when customers need constant access to their funds. The platform has spawned 35-50 internal AI applications, transforming how the company serves millions of users globally.

Tines, the workflow orchestration and automation platform for security and IT teams, built agentic workflow systems with Claude. Their agents dynamically handle workflow logic during execution, collapsing complex multi-step security operations into single-agent operations, corresponding to 100x time-to-value improvement.

And Gradient Labs, the company building custom er operations agents for financial services, deployed a customer support agent with Claude that understands customer queries within context and executes standard operating procedures. Achieving 80-90% resolution rates, their agents can handle complex workloads with limited human intervention, enabling employees to focus on relationship building and other strategic work.

AI agents open up countless possibilities for organizations of every size and sector, but implementing them requires careful consideration of architecture patterns, cost management, and operational governance.

The business case for AI agents

Think of an AI agent as a smart digital assistant that can work independently to solve complex business problems by using tools that connect to your real systems. At its core, an AI agent represents a sophisticated evolution of large language models that can autonomously direct their own processes and tool usage to accomplish complex tasks.

Traditional automation requires rigid prewritten scripts with every step mapped in advance. Agents work differently. They assess a task, choose appropriate tools, try approaches, evaluate results, and adjust strategies as needed, much like how a skilled employee tackles unfamiliar projects. For example, an agent handling customer support escalations could read the issue, check account history, consult knowledge bases, draft personalized responses, and loop in specialists, all without human intervention.

What makes these systems powerful is their capacity for autonomous reasoning and tool selection, combined with the ability to recover from errors and maintain persistence toward goal completion. Unlike traditional workflows where predefined code paths orchestrate AI interactions, agents maintain dynamic control over their decision-making processes, adapting based on environmental feedback and intermediate results.

This makes them particularly valuable for scaling complex operations where exact steps can't be predetermined, such as incident response, data analysis, customer onboarding flows, or development workflows where automated testing creates feedback loops for iterative problem-solving.

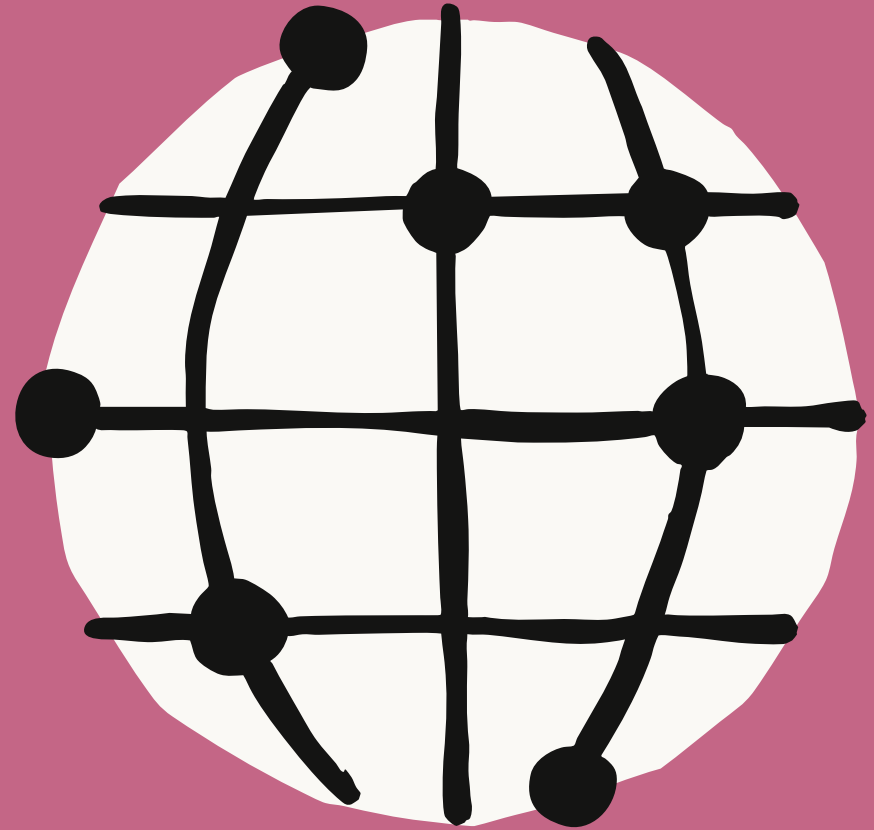
What organizations are achieving

The organizations deploying agents are seeing numbers that matter.

At a retail bank, for example, AI agents transformed credit risk memo creation. What used to take relationship managers weeks of manually reviewing ten different data sources now delivers 20 to 60 percent productivity gains and cuts credit turnaround time by 30 percent. A European equipment manufacturer with over €10 billion in revenue mapped out their agentic AI strategy and found

- from teams running these systems in production
- Architecture patterns from single agents to multi-agent orchestration, with clear guidance on matching your specific problem to the right pattern(s)
- Technical requirements including API capabilities, tool integration, and memory management for production-ready agents that actually work at scale
- Security and compliance frameworks for protecting sensitive data while managing the unique risks that come with autonomous systems
- Implementation strategies for building teams and infrastructure that scale with model improvements (instead of fighting against them)
- Future-readiness indicators to help you build systems that grow more powerful as the underlying models improve – without growing more complex

Let's dive in.



Chapter 2

Common use cases and applications for AI agents

Common use cases and applications for AI agents

To understand where agents deliver real business value, let's examine how organizations are deploying them today. Real-world implementations reveal patterns that can guide your own strategic decisions and help you identify the highest-impact opportunities within your organization.

Coding

Accelerated development on enterprise systems: Augment Code uses Claude on Google Cloud's Vertex AI to help developers navigate complex codebases with millions of interdependent lines of code. One enterprise customer completed a project in 2 weeks that their CTO estimated would take 4-8 months, while developer onboarding accelerated from weeks to 1-2 days. The platform helps teams understand sophisticated software systems faster, enabling them to write, document, and maintain code more efficiently..

Data analysis

Conversational observability data exploration: Grafana uses Claude to power an intelligent assistant that enables teams at all skill levels, from CTOs to junior engineers, to unlock observability data through natural language. Users can ask questions like "What's the request latency for my checkout service?" and Claude automatically finds relevant metrics and constructs appropriate PromQL and LogQL queries.

Customer support and operations

High-resolution automated support at scale: Intercom's Fin AI agent, powered by Claude, achieves up to 86% resolution rates with human-quality responses

across over 25,000 customers. The platform delivers a 51% average resolution rate out of the box (before customization), reduces response times from 30 minutes to seconds, and supports over 45 languages.

AI-enhanced human support coordination: Assembled uses Claude to power their Assist platform, achieving a 20% increase in customer satisfaction while decreasing support spend, over 50% reduction in escalations, and more than 30% improvement in cases solved per hour. Their approach focuses on resolving complex Tier 2+ issues rather than just deflecting simple queries.

Legal

Enterprise legal knowledge at scale: Thomson Reuters uses Claude in Amazon Bedrock to power CoCounsel, delivering expertise from 3,000+ subject matter experts and over 150 years of authoritative content to legal and tax professionals. The platform processes complex contracts and tax documents with rigorous accuracy through expert validation, with customers reporting they "can easily see it cutting the time in half, maybe even more" and describing CoCounsel's efficiency as "staggering" - freeing professionals to "focus on higher-level, more strategic work."

Flexible legal AI with superior instruction-following: Legora uses Claude to power their entire legal platform, achieving 18% higher performance on their proprietary large legal evaluation set across complex tasks. Claude Sonnet's gains come from "consistency over large tasks and documents and accurately following complex instructions," enabling Legora to build flexible agentic workflows that adapt to different practice areas and client requirements, helping lawyers "review and research with precision, draft smarter, and collaborate seamlessly."

Marketing

Automated multi-platform advertising at scale: Advolve uses Claude to orchestrate their entire digital customer acquisition process, managing millions of ads simultaneously across platforms with real-time data validation and dynamic budget allocation. The system achieves a 90% reduction in operational work time and a **15% increase in customer return on ad spend (ROAS)**, with their platform reaching "human-level ROAS when managing multi-million dollar budgets in under 30 days" for major enterprise clients managing ad budgets exceeding \$100M.

Vertical spotlight: Financial services

Automated fraud detection and risk assessment: Inscribe uses Claude to power AI Risk Agents that reduce fraud review time by 20x—from 30 minutes to 90 seconds—**while increasing output by 70x** in client examples. The AI Fraud Analyst detects fraud in images and PDFs, verifies applicant details through KYC and KYB checks, uncovers risky transactions, and provides auditable risk reports in approximately 90 seconds. This enables financial institutions to expand access to creditworthy but underserved populations including thin-file, unbanked, and credit-invisible individuals.



Chapter 3

Common architecture patterns

Common architecture patterns

These use cases demonstrate significant agent potential across industries, but successful implementation also depends entirely on choosing the right models, technologies, and an architectural approach. A customer support agent handling routine queries needs a fundamentally different design than a multi-domain research system analyzing complex datasets.

Understanding these architectural patterns helps you match technical complexity to business requirements while avoiding over-engineering that increases costs without delivering proportional value.

Let's begin with the foundational design principles that guide successful AI agent implementations.

Agent design best practices

Start simple, scale intelligently. As discussed in [Building Effective AI Agents](#), we suggest teams begin with single-purpose agents that do one thing well, then gradually develop them into more sophisticated systems as your requirements evolve. Simple systems are cheaper to run (fewer tokens, less compute), easier to debug when things go wrong, and give you clear metrics that actually tie to business outcomes.

Choose the right model for the job. There are a lot of AI models out there with a lot of different abilities and choosing the right one is critical.

The key is balancing three factors: [capabilities, speed, and cost](#). Think of it like choosing the right tool from a toolbox: you wouldn't use a sledgehammer to hang a picture frame, and you wouldn't use a tack hammer to demolish a wall. For instance, if you're building multi-agent coding frameworks or doing complex financial analysis, you'll want the [most capable model available](#). But if you're

processing thousands of straightforward customer support tickets or extracting data from forms, a lighter, faster model will get the job done just as well at a fraction of the cost.

The performance spectrum ranges from models optimized for the most complex reasoning tasks down to models designed for high-volume, straightforward applications. This means you can match your specific use case to the appropriate level of capability. Running a simple task through a premium model isn't just wasteful, it's also slower and more expensive at scale. When you're processing hundreds or thousands of requests, those differences compound quickly.

Practice modular design. This space moves [quickly; new capabilities and features emerge regularly](#). Design your systems for modularity so you can evolve your agent's capabilities without needing to radically redesign your infrastructure. Your architecture should bend with progress, not break under it.

- Agent modularity, for example, might leverage a composition pattern where:
- Prompts are defined in centralized configuration files or libraries
- Tools as discrete reusable modules

Agents are defined as needed, leveraging only the tools and/or resources needed to accomplish their assigned task.

This composition pattern might have separate modules for web search, database queries, and email composition, along with prompt templates for different reasoning styles (analytical, creative, technical) or for different roles. Following it lets you quickly define necessary agents during development and select predefined resources as needed.

Modular agents like these can scale organically, particularly those built on

frameworks like LangGraph or Mastra. As new AI capabilities emerge, a componentized agent architecture offers natural integration points without system-wide refactoring. You can easily integrate new tools into modular agent frameworks, and update the central configuration to roll out enhanced prompting techniques across all agents.

Extend capabilities with Agent Skills. Agent Skills provide a structured way to equip your agents with specialized knowledge, workflows, and tool integrations beyond their base capabilities. Rather than encoding all domain expertise directly in prompts, Skills act as modular capability packages that agents can leverage when needed.

Composable architecture: Skills can work together on complex tasks and invoke other skills as needed. This enables sophisticated workflows—for example, a compliance skill might call a document analysis skill, which in turn uses a specialized extraction skill. This composability lets you build hierarchies of capability without creating monolithic implementations.

When to use Skills:

- Domain-specific expertise (financial analysis, legal review, scientific research)
- Standardized workflows your organization has refined
- Specialized tool integrations (databases, APIs, internal systems)
- Industry-specific best practices and compliance requirements

Implementation approach: Skills integrate seamlessly with both single-agent and multi-agent architectures. In single-agent systems, Skills extend the agent's baseline capabilities. In multi-agent systems, different agents can configure different skills based on their specialization—a financial analysis agent might use risk assessment skills while a customer support agent uses CRM integration skills.

This modular approach means you can update skills independently without rewriting agent logic, share Skills across multiple Agents, and scale capabilities as your organization's needs evolve.

Build observable systems that explain themselves. AI applications often

function like black boxes, and agents add additional layers of complexity.

Beyond standard practices like structured logging, centralized monitoring, and distributed tracing, AI applications introduce unique observability challenges that traditional application performance monitoring tools weren't designed to handle.

The core issue is that AI systems are non-deterministic with opaque reasoning processes. When an AI agent fails or behaves unexpectedly, you can't simply examine a stack trace—you need visibility into prompt chains, model decision paths, retrieval contexts, token consumption, and the entire reasoning workflow. Traditional debugging often falls short when the core logic happens inside a neural network.

The key insight is that debugging AI applications requires understanding not just what happened, but why the model made specific decisions and how context flowed through multi-step reasoning chains.

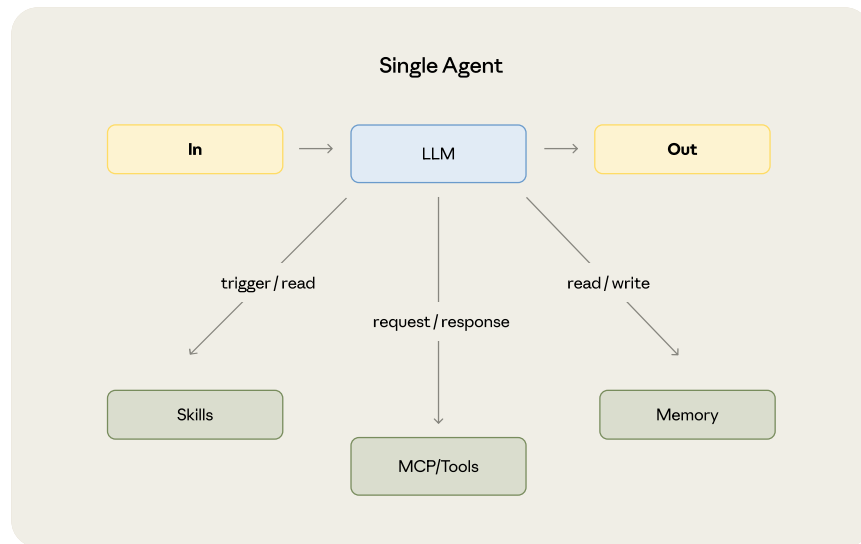
With these foundational principles established, let's examine how they apply to specific architectural patterns. We'll start with the simplest approach that works for the majority of enterprise use cases, then progress to more sophisticated patterns that justify their complexity through measurable performance gains.

Single-agent systems

In a single-agent system, an AI-powered agent operates in a continuous loop: perceive the environment, decide next steps, and act to accomplish a goal.

Typically, interacting with an agent follows this pattern:

1. A user gives the agent a task.
2. The agent then formulates a plan, executes actions based on available tools, observes the results, and adjusts its approach based on feedback.
3. The agent keeps repeating this cycle until the task is completed or it hits a stopping condition like “pause here for human review.”



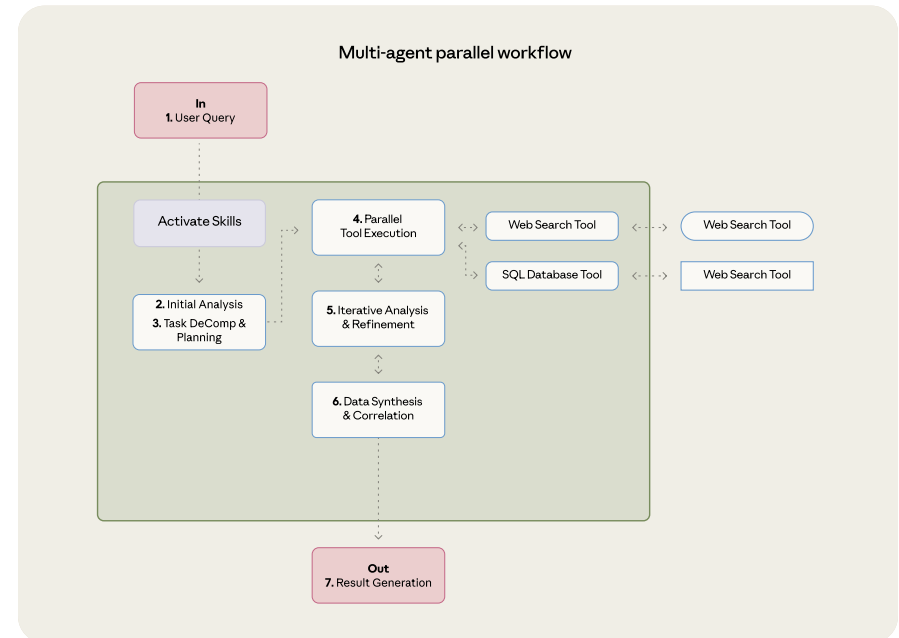
Architecture overview: The core components of a single-agent system include: an AI model that serves as the reasoning engine, a prompt that defines the agent's role and capabilities, and a toolkit of integrations that enable the agent to interact with external systems and perform specific functions. Skills also provide an additional layer of capability, equipping agents with specialized domain knowledge, workflows, and best practices that extend beyond their base training, enabling a single agent to handle complex tasks that might otherwise require multiple specialized agents.

When to use: Single agents excel when tackling open-ended problems where the path forward isn't clear from the start. You can't predetermine the solution because you don't know how many steps will be required or what obstacles will emerge along the way.

When to avoid: When you need to get the perfect answer on the first try, 100% of the time. A single agent is a powerful resource, but to handle complex problems, or to achieve the highest level of accuracy you will want to leverage multi-agent architecture. However, before scaling to multi-agent systems, consider whether adding specialized skills to your single agent might achieve your accuracy requirements more efficiently.

Example: Single-agent research agent

You deploy a research agent configured with the **Model Context Protocol (MCP)** to connect the agent to your various systems, including content repositories, business tools, and development environments. The diagram below shows how your single agent can handle a relatively complex task leveraging multiple tools.



1. User query:

An employee passes the following user query to the research agent:
"Research remote work productivity tools that engineering teams are adopting and see if any correlate with our internal productivity metrics"

2. Initial analysis:

The Claude assesses the user query:

• **Think:** "This query requires two distinct data sources: external research on productivity tools and internal company metrics. I should decompose this into parallel searches since the external research doesn't depend on internal data initially. I'll need to correlate findings at the end."

3. **Skills in action:** This research agent leverages specialized skills, including:

- **Research methodology skills** that guide systematic literature review approaches
- **Data correlation skills** that provide frameworks for identifying meaningful patterns
- **Business intelligence skills** that ensure insights align with organizational priorities

these Skills enable the agent to apply proven frameworks rather than reasoning from first principles, improving both accuracy and efficiency.

4. Task decomposition and planning:

Based on the thinking analysis, Claude plans its approach:

- External web search for productivity tool adoption trends
- Internal database queries for company productivity metrics
- Parallel tool execution for optimal efficiency
- Correlation methodology for synthesis

5. Parallel tool execution:

Claude leverages its native parallel tool calling capability to execute multiple tools simultaneously:

- Web search tool (via MCP: Searches for remote work productivity tools and adoption data
- SQL database tool (via MCP): Queries internal productivity metrics across teams and time periods

Both tools execute concurrently, reducing total response time significantly.

6. **Iterative analysis and refinement:** After processing initial results, Claude uses the think tool for deeper analysis:

- **Think:** "Web search returned comprehensive tool categories and general adoption trends, but need more specific data on engineering team preferences.

Database query successful—have baseline productivity metrics. Need refined search for quantitative adoption data and team-specific feedback."

Claude executes follow-up queries based on this analysis:

- **Web search tool:** Refined search targeting engineering-specific adoption patterns
- **SQL database tool:** Targeted queries correlating tool implementation periods with productivity changes

7. **Data synthesis and correlation:** Using the think tool for comprehensive analysis:

Think: "External research shows clear adoption trends across development tools, project management platforms, and communication systems. Internal metrics reveal productivity variations across teams and quarters. Cross-referencing implementation timelines with performance data to identify potential correlations while accounting for external factors."

8. **Result generation:** Claude synthesizes findings using extended context capabilities to maintain full conversation context and provides consolidated results:

"Research identified several categories of remote work productivity tools with significant engineering team adoption: development environment tools..."

Multi-agent systems

Multi-agent architectures coordinate multiple specialized agents to tackle complex problems that exceed the capabilities of a single generalist system. Rather than one AI model handling everything, tasks are decomposed, distributed, and executed across multiple agents, often with distinct expertise for specific types of queries. The results from multiple agents are then synthesized into a coherent response.

Internal Anthropic research shows that for complex tasks requiring pursuit of multiple independent directions simultaneously, **multi-agent systems outperform single-agent systems by 90.2%**. The key insight is that intelligence reaches a threshold where "**multi-agent systems become a vital way to scale performance**" because "groups of agents can accomplish **far more**" than individuals, much like human organizations.

Architecture overview: Multiple agents with specialized capabilities work toward common goals. This might involve orchestrators delegating to specialists or hierarchical structures where senior agents manage subagents. Communication happens directly between agents or through shared memory and message queues that coordinate their efforts. Agent Skills can also be strategically distributed across agents to create deep specialization.

When to use: Multi-agent systems excel when single agents hit fundamental limits. Choose multi-agent architectures when: (1) tasks involve open-ended problems where it's difficult to predict the required steps in advance and require the flexibility to pivot or **explore tangential connections as the investigation unfolds**; (2) you need specialized expertise that would overwhelm a generalist agent, **research shows single agents fall off sharply when there are two or more distractor domains**; or (3) problems demand **broad-based queries that involve pursuing multiple independent directions simultaneously**, where parallel processing provides substantial performance gains. They're particularly effective for complex research, comprehensive analysis spanning multiple disciplines, or scenarios requiring sustained autonomous operation across diverse knowledge domains.

Implementation considerations: Multi-agent systems deliver impressive power for complex tasks, but that power comes with proportional complexity in both architecture and operational costs. Multi-agent architectures consume tokens rapidly, requiring tasks where the business value justifies the increased performance costs. Design your system to scale effort appropriately—simple queries shouldn't trigger expensive multi-agent workflows.

Observability becomes even more critical. As outlined previously, traditional debugging approaches fail because agents make dynamic decisions and are non-deterministic between runs. In a multi-agent architecture where agent

decisions multiply, it's essential to implement tracing that captures not just individual agent behavior but agent decision patterns and interaction structures to **diagnose root causes when coordination fails**. Without comprehensive observability into how agents communicate, delegate tasks, and synthesize results, debugging becomes nearly impossible when emergent behaviors arise from complex agent interactions.

When considering multi-agent implementation, start by clearly defining what you're trying to accomplish and build the simplest solution that meets your requirements. Design for modularity and scalability from the beginning; you'll appreciate this foundation when you need to add new capabilities or scale existing ones.

Architecture patterns

Multi-agent systems organize around two fundamental coordination concepts: centralized and decentralized architectures, each addressing different coordination challenges and use cases.

Centralized systems employ hierarchical patterns where a central supervisor intelligently delegates tasks to specialized agents, creating clear chains of responsibility that mirror effective human organizational structures. These hierarchical systems are known by various names such as supervisory, orchestrator, or router patterns, with each representing slightly different permutations of centralized control, while some focus primarily on task delegation, others on routing decisions, and still others on full orchestration of agent interactions.

Decentralized systems use collaborative patterns where autonomous agents communicate directly in peer-to-peer fashion, negotiate roles dynamically, and solve complex problems through distributed intelligence. Collaborative systems are sometimes termed swarm or federated architectures, reflecting their emphasis on emergent coordination rather than imposed control.

Supporting these architecture patterns are agentic workflows, which provide structured orchestration for multi-step processes, defining the sequence and conditions under which agents execute tasks across distributed environments.

The core distinction lies in their coordination philosophy: centralized control versus distributed autonomy versus structured orchestration. Organizations frequently combine these patterns to create robust, scalable solutions that match their specific business requirements.

Hierarchical/supervisory systems

Hierarchical systems use a central controller to coordinate multiple role-specific agents through intelligent task delegation. A supervisor agent analyzes incoming requests, routes them to appropriate specialists, and synthesizes responses, creating a clear chain of responsibility that scales effectively with organizational complexity.

In hierarchical systems, individual **subagents** are treated as tools, where a supervisor agent uses a tool-calling model to decide which agent tools to invoke. This pattern mirrors how effective human teams operate: specialists focus on their domain expertise while coordinators handle task distribution and integration. Subagents can also have their own subagents, with these groups abstracted from the supervisor agent, which only interacts with the subagent team leader and remains unaware of further delegation.

The economics favor this approach despite higher token usage. While multi-agent systems consume significantly more tokens than single interactions, the performance gains justify the cost for high-value, complex tasks that require specialized knowledge or exceed single-agent context limits.

Implementation variations differ in how they balance coordination overhead with response quality:

- **Full orchestration** systems maintain complete **supervisory control over user** interactions and task execution
- **Routing-focused** implementations specialize in **delegation decisions**, potentially handing off user communication to specialist agents
- **Hybrid coordination** approaches selectively involve **supervisors based on** task complexity

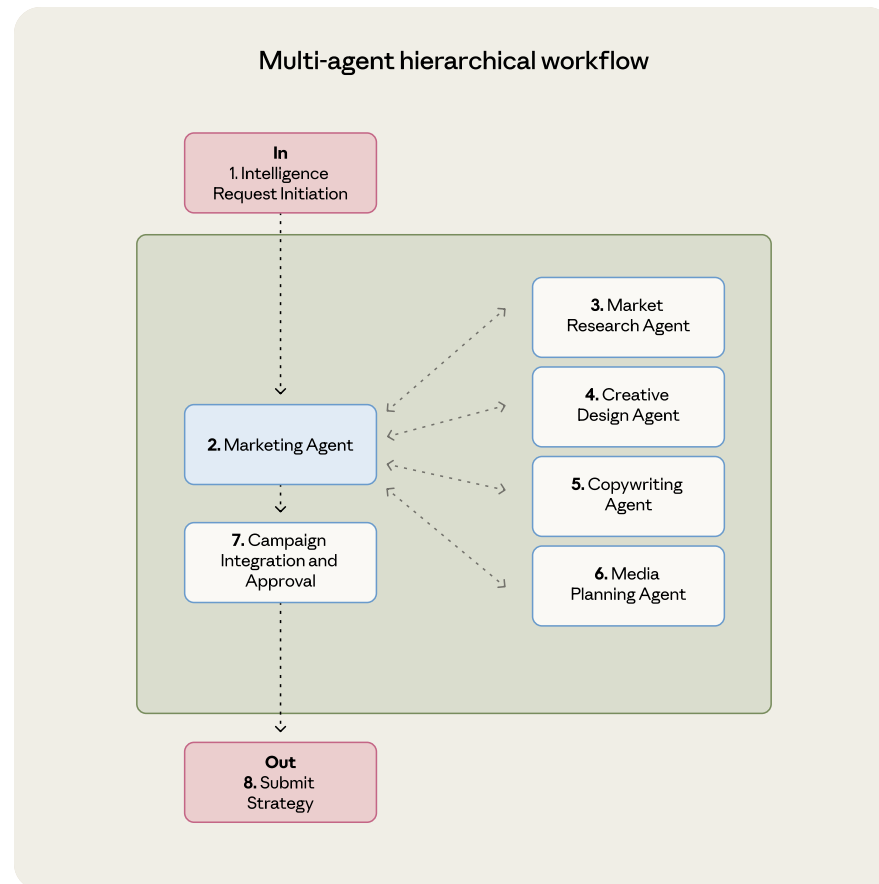
Key challenge: Context management

The orchestrator agent may face the **fundamental problem that context grows too complex for one agent to manage effectively**, creating performance bottlenecks as agents struggle to maintain coherence across extended interactions. This context complexity manifests as context window overflow, degraded reasoning performance, and coordination failures between agents.

Successful implementations need solid context management strategies: **context editing** automatically clears stale tool calls and results when you approach **token** limits while keeping conversation flow intact, and memory tools let your agents store and retrieve information outside the context window through file-based systems that **persist across sessions**. You should also consider having your tools include pagination, range selection, filtering, and truncation with sensible defaults, capping responses at manageable sizes (something like 25,000 tokens) to **prevent context exhaustion**.

Example: Multi-agent hierarchical workflow - Marketing campaign development

A marketing agency deploys a hierarchical multi-agent system to develop comprehensive marketing campaigns, with a supervisor agent coordinating specialist agents to ensure strategic alignment while leveraging deep domain expertise across all campaign components.



1. Campaign brief submission: A client submits a marketing campaign brief including objectives, target audience, budget constraints, timeline, and brand guidelines to the system.

2. Marketing director agent (supervisor): The supervisor agent analyzes the campaign requirements, identifies key deliverables, determines resource allocation, and creates a strategic execution plan that maps specific tasks to appropriate specialist agents.

3. Market research agent: Receives directive from supervisor to conduct target audience analysis, competitive landscape research, and market opportunity assessment, reporting findings back to the marketing director agent.

4. Creative design agent: Gets tasked by supervisor to develop visual concepts, brand assets, and design frameworks based on market research insights and brand guidelines, with the supervisor reviewing and approving creative direction.

5. Copywriting agent: Receives assignment from supervisor to create messaging strategy, ad copy, and content across all channels, ensuring consistency with creative direction and market positioning established by previous agents.

6. Media planning agent: Gets directed by supervisor to develop media mix recommendations, channel selection, budget allocation across platforms, and timing strategy based on audience insights and creative requirements.

7. Campaign integration and approval: The marketing director agent is tasked to synthesize all specialist outputs, ensures strategic coherence, resolves any conflicts between specialist recommendations, and prepares the integrated campaign proposal.

8. Submit comprehensive campaign strategy: The final integrated marketing campaign with creative assets, media plan, budget allocation, timeline, and success metrics is delivered to the client.

Collaborative systems

Collaborative systems enable multiple specialized agents to work together in real-time through sophisticated coordination mechanisms, sharing information and coordinating actions to achieve outcomes that exceed individual agent capabilities. Unlike hierarchical systems with central control, collaborative patterns emphasize peer-to-peer interaction where agents communicate

directly, negotiate roles dynamically, and collectively solve complex problems through distributed intelligence.

In collaborative systems, agents operate as autonomous entities that communicate, coordinate, and collaborate through various mechanisms to achieve collective goals. This approach mirrors human teamwork where specialists contribute their expertise while maintaining awareness of the broader objective. The key differentiator is that coordination emerges from agent interactions rather than being imposed by a central authority.

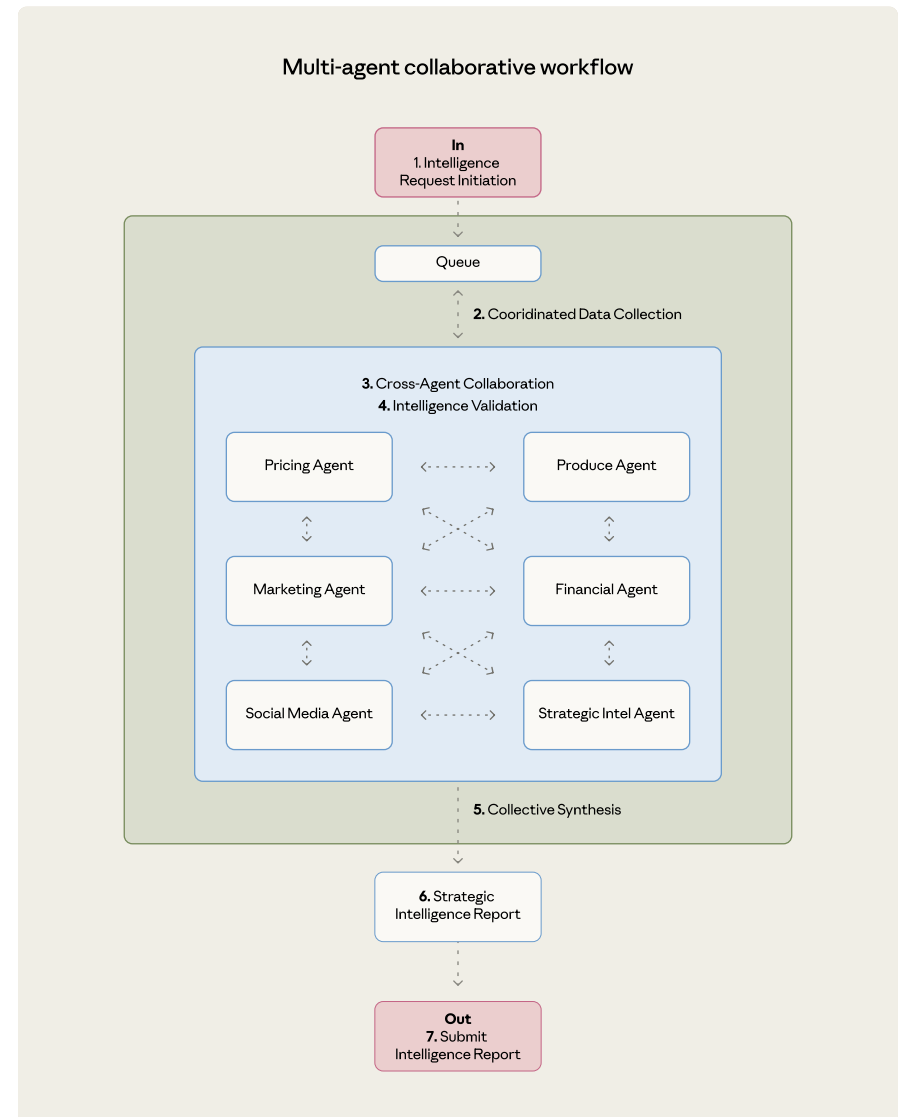
Implementation variations differ in how agents coordinate and share context:

- **Group chat orchestration** enables multiple agents to solve problems, make decisions, or validate work by participating in a shared conversation thread where they collaborate through discussion
- **Event-driven coordination** uses events as a shared language, acting as structured updates that enable agents to interpret instructions, share context, and coordinate tasks
- **Blackboard architectures** provide shared knowledge repositories where all agents can read from and write to a central knowledge repository acting as collective memory

Key challenge: Communication complexity and emergent behavior unpredictability

Collaborative systems face fundamental challenges in managing inter-agent communication and predicting system behavior. Frequent communication between agents leads to increased computational costs and complexity, while multi-agent systems have emergent behaviors that arise without specific programming, where small changes can unpredictably affect how agents behave. Success requires frameworks for collaboration that define division of labor, problem-solving approaches, and effort budgets rather than strict instructions. Additional challenges include preventing agents from bouncing tasks indefinitely and implementing robust conflict resolution mechanisms.

Example: Multi-agent collaborative workflow - Competitive Intelligence Gathering



A strategic consulting firm deploys a collaborative multi-agent intelligence system where specialized analysis agents work together in real-time, cross-referencing findings and building comprehensive competitive landscapes that exceed individual agent capabilities through collective intelligence.

1. Intelligence request initiation: Client requests comprehensive competitive analysis, triggering coordinated intelligence gathering across all specialized analysis agents.

2. Coordinated data collection: The client request is placed in a queue. Pricing, product, marketing, financial, social media, and strategic intelligence agents establish communication channels and divide monitoring responsibilities to avoid duplication.

3. Cross-agent collaboration: Agents continuously share findings in real-time - pricing agents alert product agents about feature-price correlations, marketing agents share campaign data with financial agents, and social media agents provide sentiment insights to strategic agents.

4. Intelligence validation: All agents cross-reference discoveries to identify contradictions, validate findings across multiple data sources, and build corroborated competitor profiles.

5. Collective synthesis: Agents collaborate to integrate multi-dimensional insights, assess market opportunities, and develop predictive intelligence about competitor strategic moves.

6. Strategic intelligence report: The report agent develops a comprehensive competitive landscape analysis with validated findings, predictive insights, and strategic recommendations based on collective agent intelligence.

7. Submit intelligence report: The final integrated intelligence report with metrics is delivered to the client.

Agentic workflows

Agentic workflows define the structure of how agents operate, including how they communicate, hand off tasks, and collaborate toward shared objectives. Unlike the dynamic behavior of individual agents, workflows are predefined and static. The two common agent workflow patterns are sequential and hierarchical.

Sequential workflows

Sequential workflows use predetermined control flow with defined execution paths, ensuring predictable agent transitions that are ideal for repeatable processes like document approval chains or compliance checks. These workflows provide clear audit trails and deterministic behavior, making them well-suited for regulatory environments where process consistency and traceability are critical.

Sequential workflows can leverage either software-defined decision points, such as conditional logic based on task outcomes or system state changes, or AI-driven routing where models decide application control flow based on intermediate results and contextual factors. This hybrid approach allows for both the reliability of predetermined paths and the flexibility to adapt based on content analysis or dynamic conditions.

The key advantage is operational predictability, you can map out the entire process flow, estimate execution costs, and debug issues by examining specific workflow stages. However, this predictability comes at the cost of flexibility when handling edge cases or novel scenarios that don't fit the predefined workflow structure.

When to use: Use sequential workflows when tasks can be cleanly decomposed into fixed subtasks. The main goal is to trade off latency for higher accuracy by making each AI call an easier, more focused task.

Consider sequential orchestration in these scenarios: multi-stage processes with clear linear dependencies and predictable workflow progression, data transformation pipelines where each stage adds specific value that the next stage depends on, workflow stages that can't be parallelized, and progressive

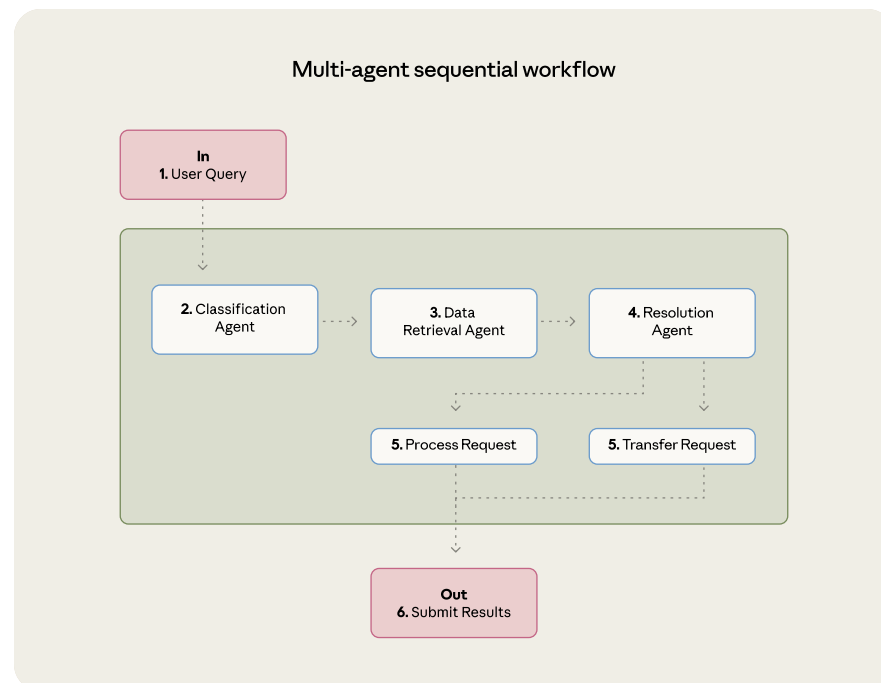
refinement requirements like draft-review-polish workflows.

Use sequential patterns when you understand the availability and performance characteristics of every agent in the pipeline, and where failures or delays in one agent's processing are tolerable for the overall task completion.

Examples where sequential workflows prove effective include generating marketing copy then translating it into different languages, writing a document outline, validating that outline meets specific criteria, then writing the full document based on the approved outline.

When to avoid: Avoid sequential workflows for processes that include only a few stages that a single agent can accomplish effectively, when agents need to collaborate rather than hand off work, or when the workflow requires backtracking or iteration.

Example: Multi-agent sequential workflow - automated data science insights



A company deploys a multi-agent workflow solution to automate their data analysis requests, enabling rapid insights generation without bottlenecking the data science team.

1. Analysis request: A stakeholder submits a data analysis request through the system (e.g., "Analyze Q4 sales performance by region" or "Identify customer churn risk factors").

2. Scoping agent: The scoping agent analyzes the incoming request, determines the analysis type (descriptive, diagnostic, predictive, or prescriptive), identifies required data sources and methodologies, assesses complexity level, and routes to the appropriate analytical pathway.

3. Data engineering agent: The data engineering agent uses the scoping output to extract data from relevant sources (data warehouses, APIs, databases), performs data cleaning and validation, handles missing values and outliers, engineers relevant features, and prepares analysis-ready datasets.

4. Analysis agent: The analysis agent takes the prepared data and executes the appropriate analytical workflow—running statistical tests, building models, generating visualizations, identifying key patterns and insights—or flags complex requests requiring human data scientist intervention with a detailed handoff package.

5. Review/escalation: The analysis results are either automatically validated and approved for distribution or queued for review by a senior data scientist for quality assurance and interpretation refinement.

6. Deliver insights: The final analysis outputs (reports, dashboards, model predictions, or recommendations) are packaged and delivered to stakeholders through their preferred channels.

Parallel workflows

Parallel workflows distribute independent tasks across multiple agents simultaneously, with results merged or processed concurrently. This pattern excels when tasks require diverse perspectives or specializations, enabling significant speed improvements through concurrent processing.

The concurrent orchestration pattern runs multiple agents simultaneously on the same task, allowing each agent to provide independent analysis from its unique perspective or specialization. This resembles the **fan-out/fan-in** cloud design pattern, where results are often aggregated but not required to be.

This pattern addresses scenarios requiring diverse insights or approaches to the same problem. Instead of sequential processing, all agents work in parallel, reducing overall runtime and providing comprehensive problem space coverage. Each agent can independently produce results within the workload, such as invoking tools or updating different data stores.

Agents operate independently without handing off results to each other, though an agent might invoke additional agents using its own orchestration approach. The pattern supports both deterministic calls to all registered agents and dynamic selection based on task requirements.

When to use: Use parallelization when divided subtasks can **be processed** simultaneously for speed, or when multiple perspectives are needed for higher confidence results. For complex tasks with multiple considerations, AI models generally perform better when each consideration is handled by a separate call, allowing focused attention on each specific aspect.

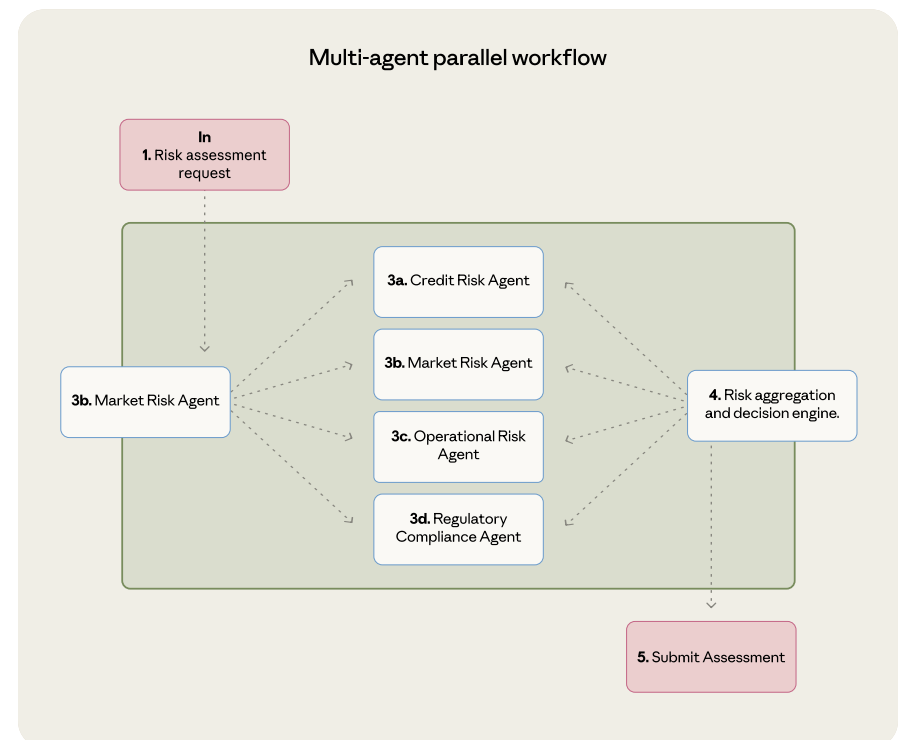
Examples where parallelization proves useful include sectioning approaches like implementing guardrails where one model processes user queries while another screens for inappropriate content, or automating evaluations where each call evaluates different aspects of model performance. Voting patterns work well for reviewing code vulnerabilities with several different prompts, or **evaluating content appropriateness with multiple prompts** using different vote thresholds to balance false positives and negatives.

When to avoid: Avoid parallel workflows when agents need **to build on each**

other's work or require cumulative context in a specific sequence, when the task requires a specific order of operations or deterministic results from running in a defined sequence, or when resource constraints like model quotas make parallel processing inefficient. Don't use parallel patterns when agents can't reliably coordinate changes to shared state or external systems while running simultaneously, when there's no clear conflict resolution strategy to handle contradictory results from each agent, or when result aggregation logic is too complex or lowers the quality of the results.

Example: Multi-agent parallel workflow - **financial risk assessment**

A financial institution deploys a multi-agent parallel workflow to evaluate loan applications and investment opportunities, enabling faster decision-making while maintaining comprehensive risk analysis across critical dimensions.



1. Risk assessment request: A loan application or investment proposal is submitted to the system for comprehensive risk evaluation.

2. Data aggregation agent: The data aggregation agent collects all relevant information including credit reports, financial statements, market data, regulatory filings, and historical performance metrics from internal and external data sources.

3. Parallel agents (leveraging their own tools)

3a. Credit risk agent: Simultaneously analyzes borrower creditworthiness, debt-to-income ratios, payment history, and collateral quality to generate credit risk scores and probability of default calculations.

3b. Market risk agent: Concurrently evaluates market volatility, interest rate sensitivity, sector exposure, and economic indicators to assess potential losses from market movements and economic downturns.

3c. Operational risk agent: In parallel, examines internal process risks, fraud indicators, compliance gaps, and operational capacity to handle the transaction, identifying potential operational failures or irregularities.

3d. Regulatory compliance agent: Simultaneously reviews regulatory requirements, anti-money laundering checks, know-your-customer compliance, and jurisdictional restrictions to ensure full regulatory adherence.

4. Risk aggregation and decision engine: All parallel risk assessments are consolidated, weighted according to institutional policies, and synthesized into comprehensive risk profiles with actionable recommendations.

5. Submit risk assessment results: The final multi-agent risk evaluation with approval/denial recommendations, risk scores, and detailed analysis reports is delivered to decision makers.

Evaluator-optimizer

Evaluator-optimizer workflows use two AI systems in iterative cycles, one generates content while another evaluates and provides feedback, repeating until quality standards are met. This pattern delivers significant improvements when properly implemented, though it comes with higher token costs.

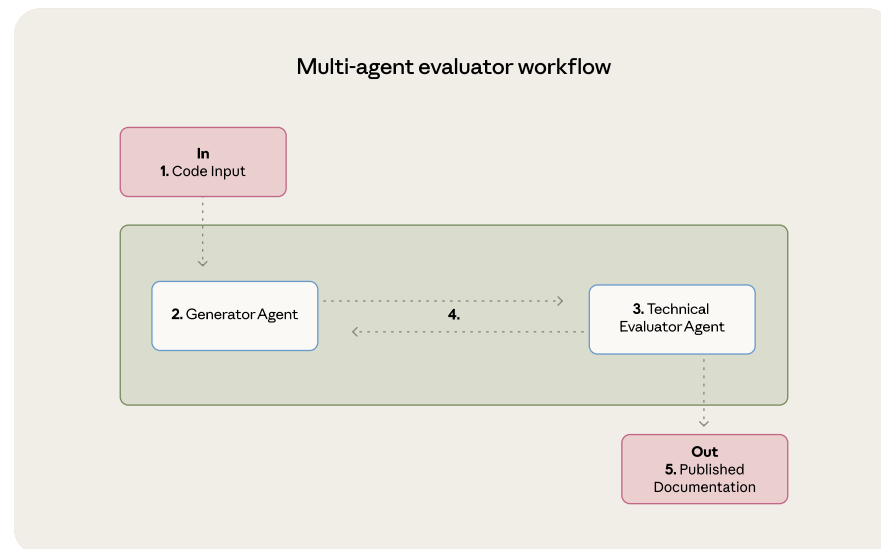
The pattern operates through structured feedback loops where a generator creates initial responses and incorporates feedback for successive improvements, while an evaluator assesses content against predefined criteria and provides actionable guidance. This resembles writer-editor collaboration, with specific suggestions incorporated in revised drafts.

When to use: Use evaluator-optimizer workflows when clear evaluation criteria exist and iterative refinement provides demonstrable value through AI feedback loops. This pattern excels for content creation requiring nuance like literary translation, code generation with security requirements, professional communications where tone matters, and research tasks needing multi-step reasoning with validation.

When to avoid: Avoid evaluator-optimizer workflows when first attempt quality already meets requirements, evaluation criteria are subjective or unclear, or when time and cost constraints outweigh quality improvements. Don't use this pattern for real-time applications requiring immediate responses, simple routine tasks like basic classification, or resource-constrained environments with strict token budgets. Avoid when deterministic solutions exist, when evaluator workflows lack domain expertise for meaningful feedback, or when performance degradation outweighs benefits.

Example: Multi-agent evaluator workflow - API documentation creator

A software development organization deploys an evaluator-optimizer workflow to automatically generate comprehensive API documentation from codebases, ensuring technical accuracy and developer usability through iterative refinement cycles that eliminate manual documentation bottlenecks.



1. Code input: Development team submits API codebase to the documentation generation system.

2. Generator agent: Analyzes codebase and creates initial documentation including endpoint descriptions, parameters, examples, and authentication requirements.

3. Technical evaluator agent: Validates documentation accuracy against actual code implementation, checking parameter types, endpoint coverage, and example correctness.

4. Refinement cycle: Generator incorporates feedback from both evaluators and iteratively improves documentation until all criteria are met.

5. Published documentation: Final polished API documentation is automatically published to the developer portal with interactive examples and comprehensive reference materials.

This process typically runs 2-4 cycles, significantly improving documentation quality while maintaining technical accuracy.

Emerging patterns

As organizations push the boundaries of what's possible with AI agents, several experimental patterns are moving from research labs into early-stage implementations. In the following section, we highlight some of these emerging patterns.

Agent pattern: Dynamic agent generation

Dynamic agent generation represents a burgeoning experimental approach that takes modularity to its logical conclusion: agents created at runtime by assembling components from libraries of prompts, tools, and configurations, then dissolved after task completion. While no production systems currently implement true dynamic creation, the technical foundations exist across multiple research projects and experimental frameworks like AutoGen or Semantic Kernel.

This pattern offers compelling advantages for resource optimization and task-specific performance; rather than maintaining pre-configured agents, systems can analyze incoming requests and automatically instantiate agents with precisely the required capabilities, then free those resources when tasks complete. However, significant challenges remain around context management complexity, emergent behavior risks, and the overhead costs of dynamic creation. Current research into multi-agent coordination and event-driven architectures provides the groundwork, but organizations should approach this as experimental territory.

Architecture pattern: Network/peer-to-peer systems

Network architectures represent significant evolution in multi-agent coordination, eliminating hierarchical bottlenecks through "many-to-many agent communication where any agent can communicate with any other agent directly". Early benchmarking shows "swarm architecture slightly outperforms supervisor architecture across the board" because agents can collaborate directly without supervisory translation layers.

Decision framework: Which pattern for which use case

Understanding architecture patterns is only the first step. The critical challenge for engineering leaders is selecting the right approach for your specific constraints: budget, timeline, complexity, and risk tolerance. Rather than choosing based on technical sophistication, successful implementations match architectural complexity to business value through systematic evaluation of three key dimensions.

The three critical questions

Before diving into specific patterns, every enterprise team needs to answer these fundamental questions:

1. What level of control do you need?

High control requirements (regulatory compliance, financial transactions, safety-critical operations) → Start with **single agents** or **sequential workflows**

Think about it this way: if you need to explain exactly why the system made a specific decision to auditors, regulators, or executives, you want predictable, traceable behavior. A single agent handling loan approvals with clear decision criteria is far easier to audit than a multi-agent system where three different AI models collaborated on the recommendation.

Moderate control requirements (customer support, content creation, data analysis) → Consider **hierarchical multi-agent systems**

For scenarios where you need flexibility but still want oversight, hierarchical systems give you the best of both worlds. A supervisor agent can enforce business rules while specialist agents handle the complexity.

Low control requirements (research, brainstorming, complex analysis) → **Collaborative multi-agent systems** become viable

When the goal is exploring possibilities or handling truly complex problems, the unpredictability of collaborative agents becomes a feature, not a bug.

2. How complex is your problem domain?

Single domain problems (answering product questions, processing returns, generating reports) → **Single agents** handle these efficiently

Don't over-engineer. If the work involves straightforward, repeatable tasks, a single well-designed agent is likely sufficient.

Multi-domain but predictable problems (employee onboarding, compliance workflows, standard analysis tasks) → **Sequential or parallel workflows**

When you can map out the process steps but need different expertise at each stage, workflows provide structure without excessive complexity.

Complex, open-ended problems (strategic analysis, research projects, system troubleshooting) → **Multi-agent architectures**

These problems require breaking down into smaller parts and different approaches. If the work benefits from multiple perspectives or **specialized skills**, multi-agent systems may make sense.

3. What are your resource constraints?

Limited budget/tokens → **Single agents** or carefully designed **parallel workflows**

Multi-agent systems use roughly 10-15x more tokens than single agents. Do the math on your expected volume before committing to complex architectures.

Time-to-market pressure → Start with **single agents**, plan an evolution path

You can deploy a single agent in weeks. Multi-agent systems take months to get right. Build something that works, then enhance it.

Long-term strategic initiative → Design for modular evolution

If this is a multi-year initiative, build your first single agent with interfaces that support adding more agents later. Design for evolution from the beginning: maintain consistent user experiences while building capability for backend architectural changes as requirements grow.

4. Do you need deep domain expertise?

Single domain with established workflows → Single agent with specialized Skills

Before jumping to multi-agent architectures, consider whether a single agent equipped with domain-specific skills can solve your problem. Skills provide deep expertise without the complexity of multi-agent coordination.

Multiple distinct domains requiring coordination → Multi-agent systems with specialized Skills

When domains must work together (e.g., legal review coordinating with financial analysis), multi-agent systems where each agent has appropriate Skills provide both specialization and coordination.

Example: A contract review system might start with a single agent using legal skills. As complexity grows, it could evolve into a multi-agent system where separate agents handle contract analysis, risk assessment, and compliance checking—each with their own specialized Skills.

Pattern selection guide

These constraints translate into clear architectural recommendations:

Single agents work best for:

- Customer service processes for well-defined product categories
- Document processing with clear business rules

- Code review and basic development tasks
- Routine analysis and reporting

Sequential workflows work best for:

- Multi-step approval processes
- Content creation pipelines (draft → review → publish)
- Data transformation and validation
- Compliance checking with multiple criteria

Parallel workflows work best for:

- Multiple perspectives improve quality
- Independent analyses can run simultaneously
- Speed matters more than coordination overhead
- Risk assessment requires diverse viewpoints

Multi-Agent systems work best for:

- Complex problem-solving requiring diverse expertise
- Research and analysis projects
- Dynamic customer interactions spanning multiple systems
- Strategic planning and decision support

Real-world evolution: An e-commerce platform's journey

- **Phase 1:** Single agent for customer inquiries (providing value)
- **Phase 2:** Routing pattern separating order status, product questions, complaints
- **Phase 3:** Specialized agents for each category with shared context

- **Phase 4:** Multi-agent system with inventory, payment, and shipping coordination
- **Phase 5:** Evaluator agents for quality assurance and continuous improvement

The key: **your architecture should evolve with your needs.** Start simple, measure everything, add complexity only when it delivers measurable value. The best architecture is the simplest one meeting today's requirements while providing a path to tomorrow's capabilities.

Hybrid architecture strategies

The decision framework provides clear starting points, but production systems often evolve into hybrid architectures that combine multiple patterns strategically. Understanding these combinations prevents architectural dead ends and enables systematic scaling.

Common hybrid patterns:

Hierarchical systems with parallel processing

- A supervisor agent delegates to specialist agents; the specialist agents coordinate parallel workflows. For example, a financial risk assessment supervisor might delegate to credit, market, and operational risk agents, each running parallel analyses within their domain.

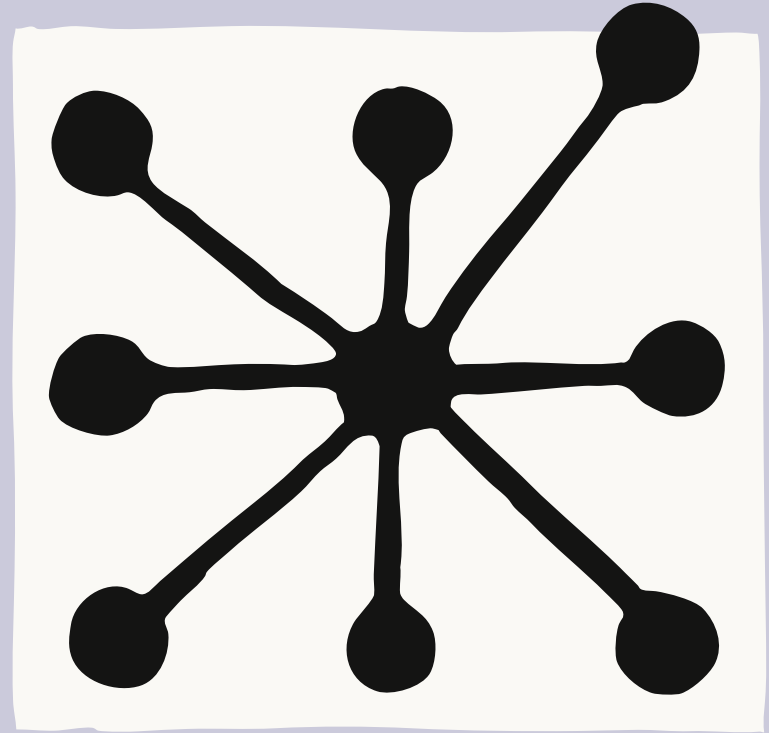
Sequential workflows with dynamic routing

- Linear processes that invoke different agent types based on intermediate results. A customer service workflow might start with classification, then route to either a simple resolution agent or a complex multi-agent research team based on issue complexity.

Single agents with multi-agent escalation

- Simple agents handle routine tasks but automatically trigger sophisticated multi-agent systems when encountering edge cases. This optimizes costs while maintaining capability for complex scenarios.

Remember, you are not bound by the simple architecture patterns. When your business needs justify the added complexity, combining patterns strategically can unlock capabilities that single approaches cannot achieve.



Chapter 4

Looking forward: the future of building AI agents

Looking forward: the future of building AI agents

In this guide we've examined the full spectrum of AI agent implementations, from single-agent systems handling focused tasks to sophisticated multi-agent architectures tackling complex, multi-domain challenges. We've seen real-world use cases across industries, explored architectural patterns and their trade-offs, and finally established frameworks for making informed implementation decisions. With this foundation you can confidently build AI agents that solve real problems and deliver real results.

Successfully implementing AI agents requires aligning technical complexity with business value rather than chasing the most sophisticated architecture you can build. You'll see the best results if you start with single agents to prove ROI, build observable systems from day one, and evolve your architecture based on what the data tells you. Organizations that take this measured approach consistently outperform those that over-engineer right out of the gate. The frameworks and patterns we've covered give you a solid foundation, but your specific implementation will depend on your risk tolerance, resource constraints, and how ready your organization is for autonomous systems.

The organization that can rapidly iterate between simple and complex approaches as business requirements evolve is the organization that can win. Whether you deploy a single customer service agent or orchestrate multi-agent research systems, your North Star must be modular designs, comprehensive observability, and clear success metrics that connect directly to business outcomes.

The tools are ready, the playbook is written. Now it's time to solve real-world problems.



Chapter 5

Next steps

Next steps

Ready to build? Get started with the Claude Developer Platform to access the models, tools, and technical documentation you need for AI agent implementation. Whether you're prototyping your first single-agent system or scaling to multi-agent architectures, these resources will accelerate your development:

- **Start building agents with the Claude Developer Platform** - Access comprehensive API documentation, implementation guides, and prompt engineering techniques.
- **Explore Agent Skills** - Learn how to equip your agents with specialized knowledge, workflows, and tool integrations.
- **Watch Building the future of agents with Claude** - Deep dive into advanced architectures and emerging pattern with the leaders who built the Claude Developer Platform.
- Explore the Anthropic Engineering Blog - Technical articles on **agent development**, **context engineering**, and **production deployment** strategies.

Contact **our Sales team** to learn more or sign up with the **Claude Developer Platform** today.



<https://www.claude.com/platform/api>